PHASE 2 — IMMUTABLE INFRASTRUCTURE · CHAPTER 07

Packer

Golden Image Creation

Blueprints, not verbal instructions — define the entire image in code so every build is identical, auditable, and reproducible.



CHAPTER GOAL

From Manual Clicks to Reproducible Code

Build automated, reproducible VM images using Packer. Instead of manually installing Ubuntu, hardening SSH, and creating templates one at a time, we define the entire image in code.

- Build identical images consistently from code
- Images are version-controlled and auditable
- Base image created once, cloned 12 times
- All security hardening is in code — reproducible, testable
- Image building is fully automated — zero manual steps

Why Packer Exists

The manual approach from Chapter 01:

1. Download Ubuntu ISO
2. Install OS manually (click through wizard)
3. Run 20+ commands to harden the system
4. Create a template VM
5. Clone it 12 times

If we need to rebuild — do we remember every command? Did we miss a hardening step?



Building a House

Manual approach:

Hire a contractor, give verbal instructions, hope they remember everything.

Packer approach:

Provide blueprints and a specification — the contractor builds it identically every time.

What Packer Does



1. The HCL template defines the entire image: builder, provisioners, variables.
2. Packer downloads the Ubuntu ISO and boots a temporary VM to install it.
3. Provisioning scripts run inside the VM: packages, hardening, cleanup.
4. Packer shuts the VM down and converts the disk into a golden QCOW2 image.
5. Both the template and the resulting image are tracked in Git for version control.

One template, one image, cloned as many times as needed — never built by hand twice.

The Building Blocks of Packer

Builder	Tool that creates images (qemu, aws, azure)	`QEMU for KVM VMs`
Provisioner	Script/tool that runs inside the build VM	`Shell scripts, Ansible`
Template	HCL file defining the entire image	`ubuntu-22.04-base.pkr.hcl`
Artifact	Output image created by Packer	`ubuntu-22.04-base.qcow2`
Variables	Configurable values in templates	`ISO URL, VM size, hostname`

Getting Packer on Your Host

Packer runs on your host machine (macOS, Linux, or WSL) — not inside a VM.

```
$ cd ~/lab

$ wget https://releases.hashicorp.com/packer/1.9.4/packer_1.9.4_linux_amd64.zip

$ unzip packer_1.9.4_linux_amd64.zip

$ sudo mv packer /usr/local/bin/

$ packer version
```



Expected output: Packer v1.9.4

Project Structure & Variables

Project Layout

```
~/lab/packer/ubuntu-22.04/
  variables.pkr.hcl
  ubuntu-22.04.pkr.hcl
  scripts/
    install-packages.sh
    hardening.sh
    cleanup.sh
    validate.sh
  output/
```

Key Variables (variables.pkr.hcl)

`iso_url`

Ubuntu 22.04 LTS Server ISO URL

`iso_checksum`

SHA256 checksum of the ISO

`vm_name`

ubuntu-22.04-base — output image name

`ssh_username / ssh_password`

Temporary credentials for provisioning

`cpu_count / memory_mb / disk_size_gb`

Build VM resources: 2 vCPU, 2048MB, 20GB

QEMU Builder Configuration

source "qemu" "ubuntu-22.04" launches a temporary KVM VM, installs the OS, and captures the result as a QCOW2 image.

Image Output

```
format = qcow2
disk_size = 20G
accelerator = kvm
```

ISO Source

```
iso_url = var.iso_url
iso_checksum =
var.iso_checksum
```

VM Resources

```
cpus = var.cpu_count
memory = var.memory_mb
```

SSH Access

```
communicator = ssh
ssh_timeout = 20m
```

`shutdown_command` triggers a clean shutdown after provisioning finishes, so Packer can safely convert the disk into a portable image.

7.4 • BUILD BLOCK

The Provisioning Pipeline

1

Stabilize

Wait 30s for the system to settle after boot

2

Update

apt-get update, upgrade, autoremove, autoclean

3

Packages

install-packages.sh — curl, git, ufw, auditd, python3...

4

Harden

hardening.sh — SSH, firewall, fail2ban, auditd, UTC

5

Cleanup

cleanup.sh — machine-id, SSH host keys, cloud-init

6

Manifest

Record build date and base OS to /etc/image-build-info

Hardening & Cleanup Highlights

hardening.sh

- SSH — PermitRootLogin no, MaxAuthTries 3, AllowUsers ubuntu
- Firewall — ufw default deny incoming, allow 22/tcp
- fail2ban — bantime 3600s, maxretry 5 on sshd
- Timezone set to UTC; auditd enabled at boot

cleanup.sh

- Remove cloud-init so it doesn't re-run oddly on clones
- Truncate /etc/machine-id — regenerated uniquely per clone
- Delete SSH host keys — regenerated on first boot
- Clear shell history, cloud-init logs, and temp files

Why cleanup matters: without it, every clone would share the same machine-id and SSH host keys — a networking and security hazard.

Validate, Build & Deploy

1 · Validate

```
packer validate .
```

Confirms the configuration is syntactically valid

2 · Build

```
packer build -on-
error=ask .
```

5–10 min: downloads ISO, installs, hardens

3 · Verify

```
./scripts/validate.sh
output/ubuntu-22.04
```

Checks size and confirms valid QCOW2 format

4 · Deploy

```
cp output/...
/var/lib/libvirt/images/
```

Moves the image to KVM's default location

If a build fails, Packer pauses and asks whether to debug (SSH into the VM) or discard it.

7.6 · VERIFICATION

Boot a VM From the Golden Image

```
virt-install \
  --name ubuntu-22.04-test --ram 2048 --vcpus 2 \
  --disk path=.../ubuntu-22.04-packer.qcow2 \
  --os-variant ubuntu22.04 --import --noautoconsole
```



Verify inside the VM

- hostname → ubuntu-22.04-test
- ufw status → active
- systemctl status ssh → active



Clean Up the Test VM

`virsh destroy ubuntu-22.04-test` → `virsh undefine ubuntu-22.04-test` → `rm .../ubuntu-22.04-test.qcow2`

Flexing the Template

Override with -var

```
packer build -var "memory_mb=4096" -var "cpu_count=4" .
```

Environment Variables

```
export PKR_VAR_iso_url="..." then packer build .
```

Multiple Variants

```
ubuntu-22.04-minimal / -server / -hardened .pkr.hcl
```

Publish to Registry

```
aws s3 cp output/ubuntu-22.04 s3://my-images/...
```

Templates Belong in Git

Commit & Tag

```
cd ~/lab/packer
git init
git add ubuntu-22.04/
git commit -m "Initial Ubuntu 22.04 template"
git tag -a v1.0 -m "First stable build"
```

What Version Control Buys You

- Track changes to the image definition
- Revert to previous versions
- Review who changed what and when
- Collaborate with teammates

Common Problems & Fixes

"qemu-system-x86_64: No such file"	*QEMU is not installed*	`sudo apt install -y qemu-system-x86 qemu-utils`
"Download failed: 404 Not Found"	*ISO URL is wrong or moved*	`Check releases.ubuntu.com; update -var "iso_url=..."`
Build hangs at "Starting VM"	*QEMU/KVM can't start*	`kvm-ok; sudo virt-host-validate qemu`
"Permission denied" on /var/lib/libvirt	*Directory needs root*	`sudo cp + sudo chown libvirt:libvirt`

7.11 · CHAPTER SUMMARY

Images First, Infrastructure Next

- Packer template (HCL) — code-based image definition
- Provisioning scripts — hardening & packages
- Automated build — no manual clicks, consistent output
- QCOW2 image — KVM-compatible artifact
- Validation script — verifies image integrity
- Git repository — version control for image definitions

NEXT: CHAPTER 08

OpenTofu

Infrastructure as Code

- All 12 VMs declared in HCL
- Network, firewall, DNS all defined
- Entire lab provisions with one command
- Destroy and rebuild in < 1 hour